Lecture 6. Relational Data

Keys, mutating joins, filtering joins, set operations

Contents

6.1 Relational data **2**
6.1.1 Keys . . . 2

6.2 Mutating joins **2**
6.2.1 Duplicate keys . . . 3
6.2.2 Defining the key columns . . . 4

6.3 Filtering joins **4**

6.4 Set operations **5**

6.5 Selected exercises **6**

6.1 Relational data

Motivation. Joins combine tables through shared variables, e.g. delays with weather.

Relational data: several tables, related in *pairs*. Besides `flights`, `nycflights13` has

- `airlines`: carrier names by code;
- `airports`: airports by `faa` code;
- `planes`: planes by `tailnum`;
- `weather`: hourly weather at each NYC airport.

`flights` links to `planes` by `tailnum`, `airlines` by `carrier`, `airports` by `origin`, `dest`, and `weather` by `origin`, `year`, `month`, `day`, `hour`.

6.1.1 Keys

A **key** is a variable (set) that uniquely identifies an observation.

- **Primary key**: in its own table, e.g. `planes$tailnum`.
- **Foreign key**: points to another table, e.g. `flights$tailnum`.

Verify uniqueness:

```
planes %>% count(tailnum) %>% filter(n > 1)
```

```
## # A tibble: 0 x 2
## # i 2 variables: tailnum <chr>, n <int>
```

```
weather %>% count(year, month, day, hour, origin) %>% filter(n > 1)
```

```
## # A tibble: 3 x 6
##    year month   day  hour origin     n
##   <int> <int> <int> <int> <chr>  <int>
## 1  2013    11     3     1 EWR        2
## 2  2013    11     3     1 JFK        2
## 3  2013    11     3     1 LGA        2
```

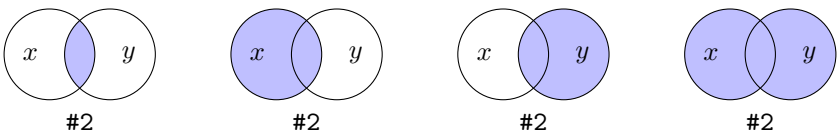
A few duplicated hours (daylight saving): `weather` is not perfectly keyed [Supplement]. Primary + foreign key = a **relation**, typically one-to-many.

6.2 Mutating joins

Motivation. Add columns from another table, matching rows by key.

Two small tables:

```
x <- tribble(~key, ~val_x,  1, "x1",  2, "x2",  3, "x3")
y <- tribble(~key, ~val_y,  1, "y1",  2, "y2",  4, "y3")
```



Inner join: pairs with equal keys; unmatched rows silently dropped.

```
x %>% inner_join(y, by = "key")
```

```
## # A tibble: 2 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
```

Outer joins keep rows in at least one table, filling `NA`:

- `left_join(x, y)` keeps all rows of `x`;
- `right_join(x, y)` keeps all rows of `y`;
- `full_join(x, y)` keeps all rows of both.

```
left_join(x, y, by = "key")
```

```
## # A tibble: 3 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     3 x3    <NA>
```

```
full_join(x, y, by = "key")
```

```
## # A tibble: 4 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     3 x3    <NA>
## 4     4 <NA>  y3
```

The left join is the usual lookup: no original observation is lost.

6.2.1 Duplicate keys

Duplicates in one table (one-to-many): the other row is copied to every match:

```
x <- tribble(~key, ~val_x,  1, "x1",  2, "x2",  2, "x3",  1, "x4")
y <- tribble(~key, ~val_y,  1, "y1",  2, "y2")
left_join(x, y, by = "key")
```

```
## # A tibble: 4 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     2 x3    y2
## 4     1 x4    y1
```

Duplicates in both: all combinations within each key, usually an error (dplyr warns of many-to-many).

```
x <- tribble(~key, ~val_x,  1, "x1",  2, "x2",  2, "x3",  3, "x4")
y <- tribble(~key, ~val_y,  1, "y1",  2, "y2",  2, "y3",  3, "y4")
left_join(x, y, by = "key")
```

```
## Warning in left_join(x, y, by = "key"): Detected an unexpected many-to-many relationship
    between `x` and `y`.
## i Row 2 of `x` matches multiple rows in `y`.
## i Row 2 of `y` matches multiple rows in `x`.
```

```
## i If a many-to-many relationship is expected, set `relationship = "many-to-many"` to
##   silence this warning.
```

```
## # A tibble: 6 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     2 x2    y3
## 4     2 x3    y2
## 5     2 x3    y3
## # i 1 more row
```

6.2.2 Defining the key columns

A narrow table shows the added columns:

```
flights2 <- flights %>% select(year:day, hour, origin, dest, tailnum, carrier)
```

- `by = NULL` (**natural join**): all common variables, reported:

```
flights2 %>% left_join(weather)
```

```
## # A tibble: 336,776 x 18
##    year month   day  hour origin dest  tailnum carrier  temp  dewp humid wind_dir
##   <int> <int> <int> <dbl> <chr>  <chr> <chr>   <chr>   <dbl> <dbl> <dbl>    <dbl>
## 1  2013     1     1     5 EWR    IAH   N14228  UA       39.0  28.0  64.4      260
## 2  2013     1     1     5 LGA    IAH   N24211  UA       39.9  25.0  54.8      250
## 3  2013     1     1     5 JFK    MIA   N619AA  AA       39.0  27.0  61.6      260
## 4  2013     1     1     5 JFK    BQN   N804JB  B6       39.0  27.0  61.6      260
## 5  2013     1     1     6 LGA    ATL   N668DN  DL       39.9  25.0  54.8      260
## # i 336,771 more rows
## # i 6 more variables: wind_speed <dbl>, wind_gust <dbl>, precip <dbl>, pressure <dbl>,
## #   visib <dbl>, time_hour <dttm>
```

- `by = "tailnum"`: that variable only. Both tables have `year` with different meanings (flight, manufacture), so a natural join is wrong; here they become `year.x`, `year.y`.
- `by = c("dest" = "faa")`: different names in the two tables; `c("origin" = "faa")` would add the origin airport.

```
flights2 %>% left_join(airports, c("dest" = "faa")) %>% select(dest, name, lat, lon)
```

```
## # A tibble: 336,776 x 4
##   dest  name                              lat   lon
##   <chr> <chr>                           <dbl> <dbl>
## 1 IAH   George Bush Intercontinental     30.0 -95.3
## 2 IAH   George Bush Intercontinental     30.0 -95.3
## 3 MIA   Miami Intl                       25.8 -80.3
## 4 BQN   <NA>                            NA    NA
## 5 ATL   Hartsfield Jackson Atlanta Intl  33.6 -84.4
## # i 336,771 more rows
```

6.3 Filtering joins

Motivation. Keep or drop rows by whether they match, adding no columns.

Top ten destinations:

```
top_dest <- flights %>% count(dest, sort = TRUE) %>% head(10)
```

- `semi_join(x, y)`: rows of `x` *with* a match; any number of keys, unlike `filter(dest %in% top_dest$dest)`.
- `anti_join(x, y)`: rows *without* a match; diagnoses mismatches.

```
semi_join(flights, top_dest, by = "dest") %>% nrow()
```

```
## [1] 141145
```

```
flights %>%
  anti_join(planes, by = "tailnum") %>%
  count(tailnum, sort = TRUE)
```

```
## # A tibble: 722 x 2
##   tailnum     n
##   <chr>   <int>
## 1 <NA>     2512
## 2 N725MQ    575
## 3 N722MQ    513
## 4 N723MQ    507
## 5 N713MQ    483
## # i 717 more rows
```

Unmatched tail numbers are mostly AA and MQ, which report fleet numbers [Supplement].

Checking a join [Supplement]

Identify the primary key from the data's meaning; check no key is missing; check foreign keys with `anti_join()`; if a left join adds rows, the key was not unique.

6.4 Set operations

Whole rows, all variables compared:

```
df1 <- tribble(~x, ~y,  1, 1,  2, 1)
df2 <- tribble(~x, ~y,  1, 1,  1, 2)
intersect(df1, df2)   # rows in both
```

```
## # A tibble: 1 x 2
##       x     y
##   <dbl> <dbl>
## 1     1     1
```

```
union(df1, df2)       # unique rows in either
```

```
## # A tibble: 3 x 2
##       x     y
##   <dbl> <dbl>
## 1     1     1
## 2     2     1
## 3     1     2
```

```
setdiff(df1, df2)     # rows in df1 but not in df2
```

```
## # A tibble: 1 x 2
##       x     y
##   <dbl> <dbl>
## 1     2     1
```

```
setdiff(df2, df1)
```

```
## # A tibble: 1 x 2
##       x     y
##   <dbl> <dbl>
## 1     1     2
```

6.5 Selected exercises

Motivation. Keys, mismatches, and when a join silently duplicates rows.

Exercise 6.1 (R4DS Exercises 13.2.1.2 and 13.2.1.3)

(a) What is the relation between `weather` and `airports`?
(b) `weather` only contains information for the three NYC origin airports. If it contained weather records for all airports in the USA, what additional relation would it define with `flights`?

Solution.

(a) Foreign key `weather$origin` → primary key `airports$faa`; one-to-many (many hourly records per airport).

(b) Destination weather at arrival: `flights$dest` to `weather$origin` with `year`, `month`, `day` and the hour of `arr_time`, a second relation besides `origin` and departure hour. ■

Exercise 6.2 (R4DS Exercise 13.5.1.5)

What do the following two anti joins tell you?

```
anti_join(flights, airports, by = c("dest" = "faa")) %>% count(dest)
```

```
## # A tibble: 4 x 2
##   dest      n
##   <chr> <int>
## 1 BQN     896
## 2 PSE     365
## 3 SJU    5819
## 4 STT     522
```

```
anti_join(airports, flights, by = c("faa" = "dest")) %>% nrow()
```

```
## [1] 1357
```

Solution. First: flights whose destination is not in `airports`: BQN, PSE, SJU (Puerto Rico) and STT (US Virgin Islands). Second: 1357 airports received no NYC flight in 2013. So `flights$dest` is not fully matched, while unused rows of a lookup table are normal. ■

Exercise 6.3 (how many rows a join returns)

For a key value k let a_k and b_k be the numbers of rows of `x` and `y` with that key.
(a) Show that `left_join(x, y)` has $\sum_k a_k \max(b_k, 1)$ rows, and hence exactly as many rows as `x` if and only if no key value of `x` occurs more than once in `y`.
(b) Use (a) to explain the check below.

```
planes %>% count(tailnum) %>% filter(n > 1) %>% nrow()
```

```
## [1] 0
```

```
nrow(left_join(flights, planes, by = "tailnum")) == nrow(flights)
```

```
## [1] TRUE
```

Solution [Supplement].

(a) One row per matching pair, one `NA` row per unmatched row of `x`: key k gives $a_k b_k$ rows if $b_k \geq 1$, a_k if $b_k = 0$, i.e. $a_k \max(b_k, 1)$; sum over k. The excess over $\sum_k a_k$ is $\sum_k a_k\{\max(b_k, 1) - 1\} \geq 0$, zero iff $b_k \leq 1$ for every key of `x`.

(b) No tail number repeats in `planes` (a primary key), so by (a) the row count is kept, as line two confirms; a duplicate would silently copy that plane's flights. ■